

An Investigation into using LLMs for modelling outbreaks of Ebola

Introduction

AI is currently used in healthcare settings in low-risk situations, such as streamlining administrative tasks and pattern recognition in scans. The ability of AI to access and process vast amounts of data, as well as literature discussing key epidemiological features of diseases, has the potential to inform timely and targeted responses to health concerns and increase the efficiency of providing public health guidance. However, there are significant challenges associated with integrating the use of AI in healthcare settings. Besides issues with financing and deploying AI usage, it is important to consider the reliability of the implementation to mitigate risks including potential AI hallucinations. The use of AI should be constantly evaluated and managed to improve current healthcare systems (BMA, 2024).

This eight-week research project focused on developing an understanding of the ability of Large Language Models (LLMs) to produce code to simulate and fit compartmental models, in particular for Ebola given the recent outbreak in the Democratic Republic of Congo and Uganda. Thus, we investigated the suitability of code generated for simple compartmental models, the relevance of changes made to the code to tailor the model for an Ebola outbreak, and the effect of prompting for some other forms of additional complexity. We then attached academic literature to the prompts to determine the level of consistency the generated model code had with the topics discussed in the papers.

The project has been completed in collaboration with the UKHSA, a government organisation which aims to prevent and reduce harm to people from various health threats (GOV.UK, 2021). The UKHSA make use of predictions from mathematical models to inform decisions. The rest of this case study discusses the suitability of the implementation of some LLMs to this aim.

Motivation

In the face of disease outbreaks, reliable predictions for future dynamics of the spread of infection are key to making informed decisions. Having an idea of the expected peak numbers of infections or the timing of this peak can help governments to prepare resources and implement interventions. This means healthcare systems are better equipped to cope with the strain when this happens, and resources aren't wasted when and where they are less required (Gao et al., 2025).

Predictions produced by different models for the spread of infection can be weighted together to produce an ensemble, see Figure 1. It has been found that predictions from these multi-model ensembles are often more reliable and accurate. Therefore, quickly generating this joint model from the collection of singular models is of interest so in turn better informed decisions can be made quicker (Shandross et al., 2026).

We aimed to investigate whether we can depend on the use of LLMs to speed up the ensemble generation process. We have been interested in the ability of different LLMs to generate compartmental model code for disease spread models found in academic literature, evaluating how well the generated code aligns with the discussions of the attached literature and assessing the reliability of the generated output. If this process is found to be successful, the code for single models can be generated quicker, and hence aggregated to produce an ensemble more efficiently.

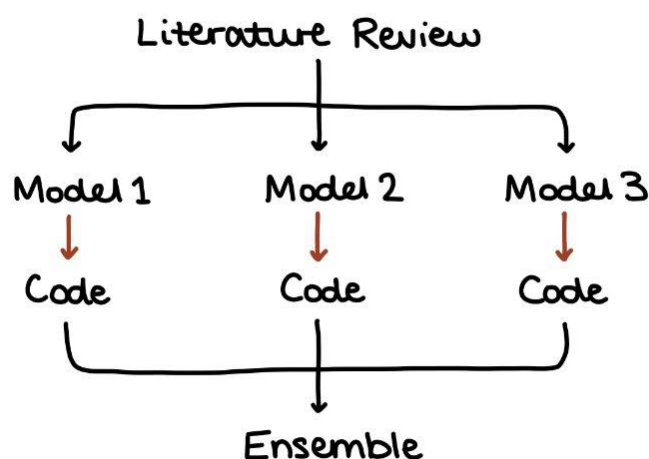
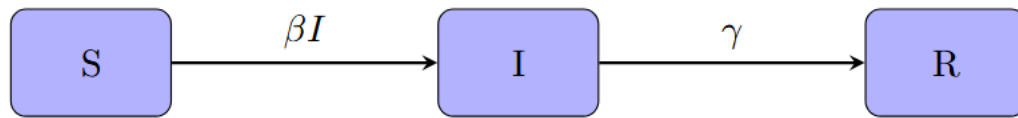


Figure 1: Producing an Ensemble

Model Structure

Compartmental models are often used in epidemiology. These mathematical models allow us to predict the dynamics of population epidemics from just current information on individual epidemiological factors, such as transmission rates. We can also then consider long-term characteristics from early dynamics and the potential impacts of interventions. The population is split into distinct groups, each described by an individual's current disease status. The movement of individuals between compartments is modelled by a system of ordinary differential equations (ODEs). The solutions to these equations, found via numerical methods, provide the number of individuals in each compartment at each time

step. These disease trajectories are used for predictions which then help to construct informed decisions (Keeling and Rohani, 2008).



$$\begin{aligned}\frac{dS}{dt} &= -\beta SI \\ \frac{dI}{dt} &= \beta SI - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

Figure 2: State transition diagram and ODEs for SIR model

The SIR (Susceptible – Infected – Recovered) model, Figure 2, splits the population into 3 compartments and is used in epidemiology for modelling the disease dynamics in this simple case. Individuals in the population are categorised as susceptible (able to be infected by the pathogen), infected or recovered (cleared the pathogen and received immunity). The system of ODEs here describes the movement from compartment S to I and from I to R, where:

- β is the rate of transmission (inverse of incubation period);
- γ is the rate of recovery (inverse of infectious period);
- βI is the force of infection.

The force of infection is defined as the rate (per capita) at which susceptible individuals become infected. Here, the force of infection is proportional to the number of individuals currently infected in the population.

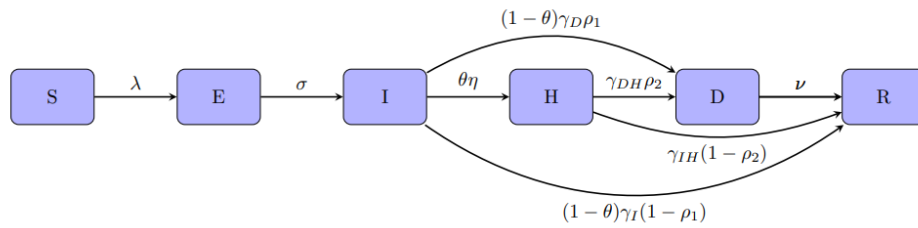
Additional complexity can be added to compartmental models by introducing:

- More compartments;
- Stochastic dynamics;
- Time-varying transmission;
- Spatial spread of disease;

- Waning immunity;
- A demographic population.

To construct a compartmental model specified to a particular disease we need to consider the key epidemiological features of that disease. Here we show reasoning for the structure of a commonly used model for Ebola (Legrand et al., 2006).

Ebola is a highly infectious virus which is spread through direct transmission. This is reflected in the model by high rates of transmission. An infected individual is not infectious for a period before becoming symptomatic, estimated at 2-21days (WHO, 2025). We represent this incubation period in the model using an exposed compartment, Figure 3. Isolation in hospital is the usual treatment for Ebola, where individuals are kept hydrated and quarantined to reduce transmission. Ebola infected individuals who died from the infection remain infective for a time. This means that recently deceased individuals contribute to transmission, and removal from this group is modelled by a rate of safe burial. Splitting the infectious population into the infected in community, infected in hospital, and infected deceased allows us to consider the differing transmission rates between these groups. Waning immunity and the effects of a demographic population are not represented here, since Ebola outbreaks are often too short to be influenced by these effects, and the infection mortality too great in comparison.



$$\lambda = \frac{\beta_I I + \beta_H H + \beta_D D}{N}$$

$$\frac{dS}{dt} = -\lambda S$$

$$\frac{dE}{dt} = \lambda S - \sigma E$$

$$\frac{dI}{dt} = \sigma E - \left[\theta \eta + (1 - \theta) [\gamma_I (1 - \rho_1) + \gamma_D \rho_1] \right] I$$

$$\frac{dH}{dt} = \theta \eta I - \left[\gamma_{IH} (1 - \rho_2) + \gamma_{DH} \rho_2 \right] H$$

$$\frac{dD}{dt} = (1 - \theta) \gamma_D \rho_1 I + \gamma_{DH} \rho_2 H - \nu D$$

$$\frac{dR}{dt} = (1 - \theta) \gamma_I (1 - \rho_1) I + \gamma_{IH} (1 - \rho_2) H + \nu D$$

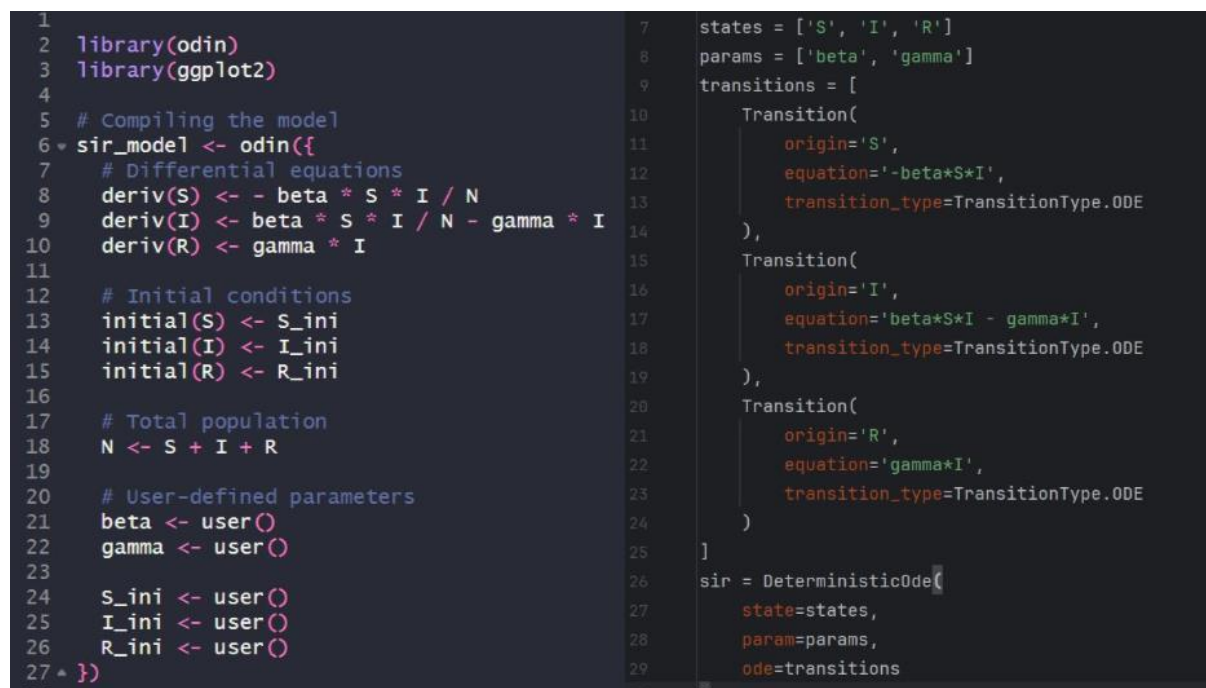
Figure 3: State transition diagram and equations for SEIHDR model

Code Representation of Compartmental Models

Disease dynamics are modelled by a complicated system of ODEs, and the solution to this involves numerical approximations. In the code generation portion of this project, we have used Odin and PyGOM in R and Python respectively to find the solutions to these equations, Figure 4.

Odin is a Domain Specific Language (DSL) in R for describing and solving ODEs. A particular model is represented by its state variables and their initial conditions, updating equations and the parameters. This DSL compiles the code to C code, speeding up simulations (GitHub, 2026).

PyGOM is a package in Python for building and solving ODEs, with a focus on epidemiological models. The transitions between compartments can currently be specified in 4 different ways, including the full ODE systems and just the transition between states (Tye et al., 2018).



```
1 library(odin)
2 library(ggplot2)
3
4 # Compiling the model
5 sir_model <- odin({
6   # Differential equations
7   deriv(S) <- - beta * S * I / N
8   deriv(I) <- beta * S * I / N - gamma * I
9   deriv(R) <- gamma * I
10
11   # Initial conditions
12   initial(S) <- S_ini
13   initial(I) <- I_ini
14   initial(R) <- R_ini
15
16   # Total population
17   N <- S + I + R
18
19   # User-defined parameters
20   beta <- user()
21   gamma <- user()
22
23   S_ini <- user()
24   I_ini <- user()
25   R_ini <- user()
26 })
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figure 4: SIR code using Odin (left) and PyGOM (right)

Fitting Compartmental Models to Data

We were also interested in a framework to compare the different compartmental models we attain and consider whether the LLMs can successfully perform this important stage of model selection. Fitting these models to data helps to provide reasonable estimates for

parameters and decide to what extent they seem to produce accurate realisations of real-world data (Martcheva, 2015).

The data we have is often assumed to be part of a time series. The process of fitting a mathematical model to data is described in Figure 5.

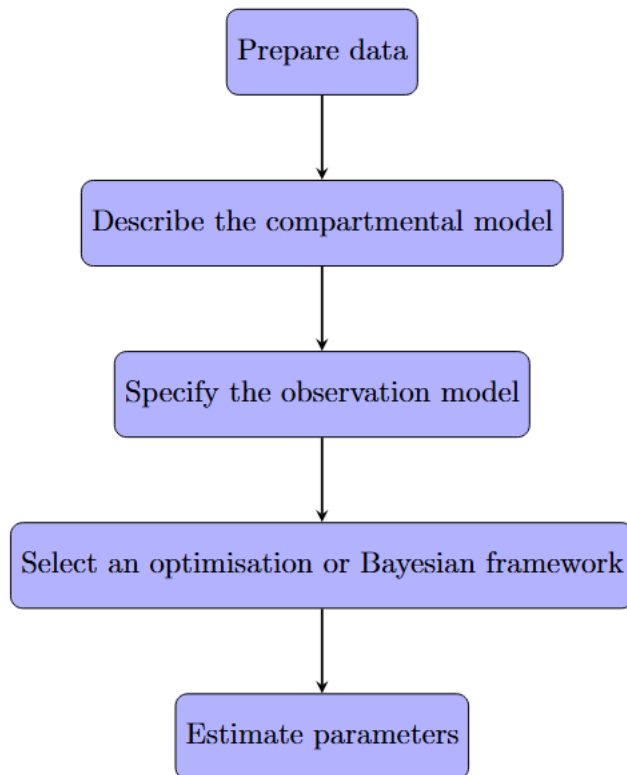


Figure 5: Fitting compartmental models to data framework (Hoang and Ehrhardt, 2025)

1. Prepare the data – clean the data so it is suitable for model fitting.
2. Describe the compartmental model - define the system of ODEs for the disease dynamics being modelled, specifying the parameters to be estimated and those to be fixed to literature values.
3. Specify the observation model - define a count-based model to link the data to the output.
4. Select an optimisation or Bayesian framework.
5. Estimate parameters - Numerically solve the model iteratively until convergence is achieved.

The compartmental model supposedly describes the true disease systems, but real-world data is often noisy, and we want to limit the effects of under-reporting. Hence, we introduce an observation model to capture more uncertainty in the model. This observation model

shows how the outputs of the compartmental model relate to the real-world data through a defined statistical distribution (Domenech de Celles and Kramer, 2026). The observation model describes how the data related to the compartmental model output and is used in defining the likelihood, which is used in optimisation and Bayesian frameworks (Tang et al., 2020).

Given our interest here in the number of cases or infections, we would like to make use of an observation model for count-based data. A Poisson observation model is a natural consideration here, however with the variance set equal to the mean we have no free dispersion parameter. A Negative-Binomial observation model is often preferable, since the extra dispersion parameter helps to model situations with substantial overdispersion. This is often the case in epidemics, where under-reporting leads to many zero cases and extreme spikes.

In the Frequentist framework, an objective function is optimised to find a single best fit of parameters. This may involve minimising the sum-of-squares error in Least Squares Estimation (LSE) or maximising the likelihood in maximum Likelihood Estimation (MLE).

In the Bayesian framework, the whole posterior distribution is characterised, so multiple possible parameter estimates are provided in proportion to how well each fits to the data (Pourzanjani, 2018). Prior distributions for parameters are specified as well as the observation model, and the posterior distribution is found using Bayes' Theorem. Approximate samples for this posterior distribution can be generated using methods such as Markov Chain Monte Carlo (MCMC). Prior and posterior predictive checks can be used to check the validity of the priors and model.

Stan is a probabilistic programming language that was designed for Bayesian inference and has been used in this project to fit Bayesian compartmental models to data. It supports probability distributions, matrix operations and ODE solvers and uses Hamiltonian Monte Carlo (HMC), a Markov Chain Monte Carlo (MCMC) method (Grinsztajn, 2021).

Model Comparison

Once the model has been fit to data, the validity of the model can be checked using metrics which assess forecasting accuracy and by testing forecast periods.

Examples of the forecast accuracy metrics are Mean Absolute Error (MAE), Root Mean Square Error (RMSE) and coverage of prediction intervals (Eddin et al., 2025).

The MAE measures the average difference between the predicted and actual number of counts. Therefore, a smaller MAE represents a model with more accurate forecasts.

The RMSE is the square root of the average squared difference between the predicted and actual number of counts. A lower value again indicated better forecasting ability. The RMSE can be calculated at regular intervals over time so the forecasting accuracy can be documented over the course of the outbreak.

Cross-validation estimates prediction error by training (i.e. fitting) the model using a portion of the data and testing the forecasting ability of the model against the remaining data (Bates et al., 2021).

Language models and resources used

This project intended to compare multiple LLMs and interfaces they are accessed through.

We have made use of 2 Copilot interfaces: GitHub Copilot through Visual Studio Code (using an education account) and the Copilot chat function through Microsoft 365.

The Microsoft 365 Copilot gives the option for different GPT models as well as an auto function where the Copilot selects a model and length of time to think itself. Originally, the GPT models available were 5.2 or 5.5 with Quick Response or Think Deeper functions in both. As the project progressed, this updated to only 5.5 Quick Response and 5.6 Think Deeper. During the discussion below, it has been specified which model has been used for the code output for completeness.

The chat function used through Visual Studio (VS) Code selected a model of its own choice, providing the model it has used at the end of each response. This was generally MAI-Code-1-Flash, Claude Haiku 4.5 or GPT 5.4 mini. The VS Code interface was particularly useful for this project's aim. The integrated chat panel and workspace made code generation and debugging more convenient. Copying code between other AI interfaces and the desired Integrated Development Environment (IDE) was much more cumbersome.

Additionally, some basic prompting was provided to an LLaMA vllm to investigate the code generated by this open-source AI. The prompting was completed through a Pi assistant, and the LLM used was the Qwen 3.6 35B A3B FP8 model. We installed the pi-hermes-memory package to allow the LLM to learn from previous prompting and outputs.

With the aim of fitting the compartmental models to data, this project made use of the WHO GitHub repository for the current outbreak of Ebola (2026 Bundibugyo Ebola outbreak), (INRB-UMIE, 2026).

Results and Discussion

Initially, prompts were provided for generating compartmental models for Ebola without attaching literature. Here, we aimed to compare success across programming languages and investigate the ability of different LLMs to incorporate other concepts into the code.

Generic Compartmental Models

We began by considering the consistency of the models produced here in Odin and PyGOM in comparison to hand-coded models, and between the 3 LLM interfaces.

Example Prompt:

Code up a compartmental model for the disease spread of Ebola using
R/Python/Odin/PyGOM

All 3 interfaces successfully generated code for SIR models. The GPT models (5.2 and 5.5) through Microsoft 365 generated Odin and PyGOM code with correct syntax and documentation. The VS Code MAI-Code-1-Flash LLM needed the prompt to specify which PyGOM classes to use (SimulateODE etc.), otherwise the code would call outdated ones, no longer the recommended API (e.g. from pygom import Model). The code given also generally included a SciPy alternative as a fallback. Further PyGOM code through the VS Code interface was written more effectively due to LLM access to the existing workspace which had correctly formed code blocks. The Isambard vllm initially uses strange syntax, not in line with current Odin or PyGOM documentation. This LLM learns from prompts describing the issues and searches current documentation for the format desired for the code to be written in, Figure 6. The code subsequently generated consistently then reproduces the desired format.

```

7
8 = parameters {
9   beta : # transmission rate
10  gamma : # recovery rate
11  N : # total population
12 }
13
14 = state {
15  S : # susceptible
16  I : # infected
17  R : # recovered
18 }
19
20 = init {
21  S <- N * 0.999
22  I <- N * 0.001
23  R <- 0
24 }
25
26 = derivatives {
27  dS <- -beta * S * I / N
28  dI <- beta * S * I / N - gamma * I
29  dR <- gamma * I
30 }
31
32 include "odin_base.odin"

sir_odin <- function(beta = 0.3,
                      gamma = 0.1,
                      S0 = 999000,
                      I0 = 1000,
                      t = 0:200) {

  model <- odin({
    deriv(S) <- -beta * S * I / N
    deriv(I) <- beta * S * I / N - gamma * I
    deriv(R) <- gamma * I

    initial(S) <- S0
    initial(I) <- I0
    initial(R) <- 0

    N <- S0 + I0

    beta <- user(0.3)
    gamma <- user(0.1)
    S0 <- user(999000)
    I0 <- user(1000)
  })

  instance <- model$new(beta = beta, gamma = gamma, S0 = S0, I0 = I0)
  as.data.frame(instance$run(t = t))
}

```

Figure 6: Code using Odin produced by Isambard vllm, initial incorrect use (left) and corrected use (right)

Coding Models for Outbreaks of Ebola

Example prompt:

Code an SIR model using R/Python/Odin/PyGOM

-> Extend this SIR model to model key aspects of an Ebola outbreak

Prompts had to specifically mention the disease aspects desired for the model (e.g. hospitalisation, rate of burial) for the Github Copilot in VS Code (using MAI-Code-1-Flash) to code a model better tailored to Ebola than an SEIRD model.

The GPT models through Microsoft 365 tended to suggest full SEIHFR models, but an SEIRD model was once suggested as a “minimal teaching model”. This simplified model may have been given due to excessive previous prompting. Code was commonly given for plotting the compartment trajectories, Figure 7. The GPT 5.5 Think Deeper model provided an Ebola outbreak SEIHFR model when extended from an SIR model. This model was then prompted with some properties of Ebola (e.g. funeral transmission, incubation period, hospitalisation); the LLM recognised the symptoms, transmission and treatments as Ebola-like and provided an SEIHFR model. In this case the E compartment was split into 3 for early, mid and late incubation. These comments indicate the necessity of prompts in certain LLM interfaces to specify the key epidemiological features for inclusion, or the parameterisation desired for the system.

The Isambard vllm successfully extended the SIR model to incorporate separate epidemiological features, for instance an incubation period. Despite a correct discussion of the key epidemiological features of Ebola, the vllm here struggled to fully express these in

the model. Further prompting rectified this. The issues were resolved by introducing a burial rate to remove individuals from the D compartment and by defining the parameters leaving the I and H compartments in terms of fatality probabilities and overall removal rates, ensuring that units were consistent.

Once the model code has been generating with the expected compartments the predicted trajectories from the generated SEIHFR models are consistent across models and programming languages. The only differences found are due to different definitions of the full population. The infective deceased individuals may be included in this count, whereas other models describe the full population as only the full living population. This difference in population description occurred in both Copilot interfaces used.

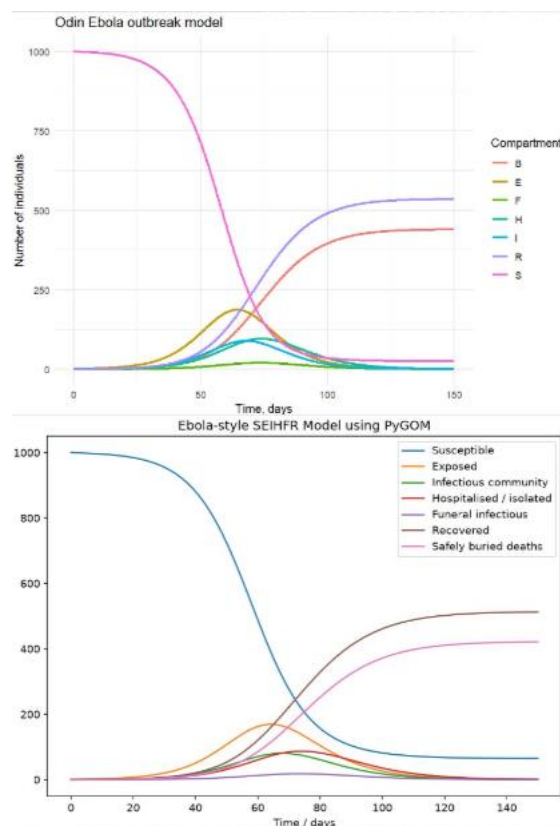


Figure 7: Trajectories from code generated using Odin (above) and PyGOM (below)

To investigate consistency of code output, the LLMS were prompted with the R variation of the initial Ebola model prompt twice. The GPT 5.5 Think Deeper through Microsoft 365 provided consistent model compartments, but the model was parameterised differently, with Case Fatality Ratios (CFR) being used once, and death rates the next. The responses through VS Code and the Isambard vllm were more consistent than this, due to the influence of code in the current workspace and the memory package respectively.

Model Generation by Location

We investigated whether the LLMs would be explicit about what should be included in an Ebola Compartmental model by country. The code generated was generally suited to an Ebola outbreak (i.e. SEIHFR) but was not greatly adjusted for locations, particularly for the VS Code Copilot models. The initial conditions were occasionally anchored to the beginning of a known outbreak in that country, and the response from GPT 5.5 Think Deeper through Microsoft 365 modelled the UK version as having lower transmission in hospital, faster burials, and the initial case being imported. However, this same model also gave the DRC system a vaccinated compartment, despite there currently being no vaccine available for the strain found in the DRC.

The code output from MAI-Code-1-Flash through the VS Code Copilot was even less specified by country, again due to the influence of current code in the workspace.

Fitting Compartmental Models to Data

The ability of LLMs to fit their generated models to data was assessed using Copilot LLMs through Microsoft 365 due to limited AI credits with the GitHub Copilot account. We made use of the GPT models (5.5 Quick Response and 5.6 Think Deeper) as well as comparing these to the auto model responses.

How should data be provided?

The LLMs couldn't directly retrieve CSV contents from the link to the GitHub repository. The models were able to process and analyse data from copied tables of data and attached downloaded CSV files.

Initially, the data provided consisted of new daily incidence of cases or deaths. Days with no new incidence were not present in the table, and it was necessary to instruct the LLM to include these days with a 0 value or else these would be disregarded.

When prompted to produce time series lists for cumulative case and death data, the LLM produced datasets, but again the dates which didn't have any data were misinterpreted. Here, they were given a 0 value rather than continuing the cumulative trend. Once these 0 spikes were fixed, the cumulative datasets were successfully converted to daily cases and deaths.

What were the differences between LLMs?

The GPT 5.6 Think Deeper spent a significantly longer amount of time to produce output for fitting models to data than alternative LLMs. This was accentuated when the prompt vaguely described the method required. It was found that prompting for generating code

without implying for the LLM to find this fit itself helped to reduce the time taken for code generation. The GPT models provided more appropriate constraints for the chosen optimisation method, for instance, by specifying bounds for L-BFGS-B. Compared to the GPT 5.5 Quick Response, the GPT 5.6 model interspersed more sanity checks in between large chunks of code as well as giving more description about limitations of methods. The GPT 5.5 model included all aspects prompted for and responded faster than the 5.6 model. However, this sometimes came at the expense of simplifying the specified coding methods (e.g. using deSolve instead of Odin).

The GPT models were more likely to follow through on their recommendations provided in discussion sections of output. This includes making use of a negative binomial observation model, whereas the auto model was seen to highlight this model as the most suitable but use a Poisson model in code for simplicity. The auto LLM was however able to provide correct code for both observation models used for count data when explicitly prompted to use each.

Furthermore, GPT models were more likely to include a reporting probability in their models. Auto may do in subsequent prompts, but it was often an extension rather than initially coded for. The auto function provided initial conditions inconsistent with the data, such as unattainable values for the initial population number. These comments highlight the need to give the auto model more context and guidance for the method of interest.

Can the models fit compartmental models to data using both Bayesian and Frequentist methods?

The LLMs can provide code for both frameworks, favouring use of rstan in the Bayesian framework. Without any specific prompting, all models use a Frequentist framework. Specifically, for the auto LLM this was often by maximum likelihood estimation but was also frequently by least-squares. The GPT models use least-squares or other less frequently, mainly making use of MLE.

The GPT 5.6 model estimated parameters entirely by summarising the data and using estimations such as empirical growth rate. This did not involve an optimisation method and occurred when the LLM was prompted with 2 forms of data, the daily cases and daily deaths. Hence, when prompting uses multiple datasets, it is best to specify the method of estimation to be used.

As discussed, the LLMs could provide code for both estimation frameworks. The use of observation models in the process of fitting models to data was often mentioned unprompted as a good modelling decision. Output considering Frequentist methods were frequently well specified. Different optimisation algorithms were used for fitting the models using MLE: L-BFGS-B, BFGS, Nelder-Mead.

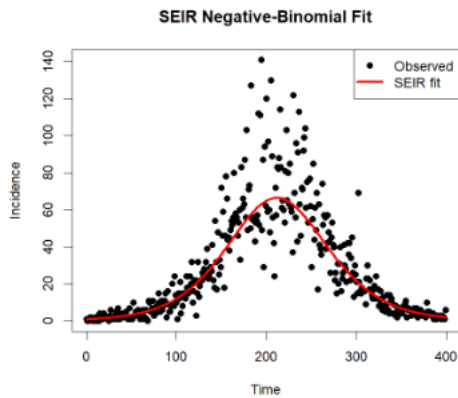


Figure 8: SEIR model fitted to simulated data using MLE

However, more syntax issues were found in Bayesian coded solutions. The issue was mainly having code generated incompatible with current syntax for stan. GPT models' output was more reliably consistent with current rstan syntax. Specifying the rstan version in use allowed the code provided to be in line with current syntax. It was also of use to prompt how the rstan code should be included within the R code: within the R file or in a separate stan file.

The LLMs tended to use lognormal, exponential or beta distributions for priors. The prior distributions were often well chosen, with positive distributions given for rate parameters and beta distributions for probabilities or proportions. However, the parameters chosen for these priors often lead to trajectories prone to explosion or unreasonable compared to the disease trajectory in the data. Care should be taken to ensure priors aren't too uninformative.

When coded, for the LLMs produced code for performing prior and posterior predictive checks. This highlighted the issues with the chosen prior distributions. Some code for prior predictive checks didn't make use of the priors at all. This was improved by prompting the model to use the generated parameters block within rstan.

The LLMs could produce code to find RMSE and coverage for many different compartmental models and fitting methods. The RMSE was given as a single value, separately for cases and deaths, and a combined value.

Summary:

Overall, the Copilot LLMs through Microsoft 365 are able to fit compartmental models to data. The ease of code generation and debugging is improved when prompts specify the framework for fitting, programming tool versions and observation model required.

Despite this seeming success, models often fit too many parameters than can be identifiable – leading to poor scaling of fits. Models were also seen to fit too few parameters to successfully describe the disease trajectory, see Figure 9. Many different parameter combinations can generate almost identical incidence curves. The fits found are often just one of these curves, and parameter estimates could be very far from true. As such, care should be taken to interpret the estimated parameters correctly and check the validity of the fitted model.

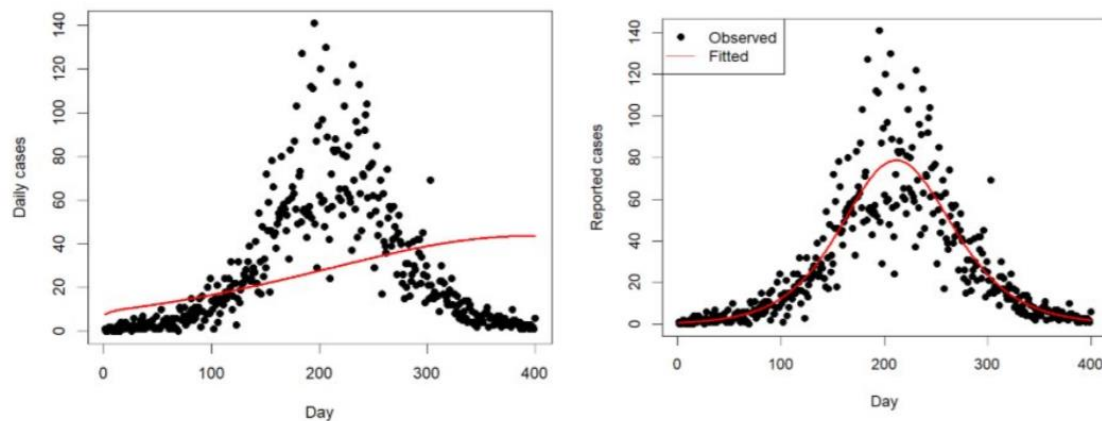


Figure 9: SEIHR model fitted to simulated data, initially only estimating 3 parameters (left), plot after prompted to estimate all transmission parameters and the reporting rate (right)

Generating Code with Attached Literature

The LLMs found through VS Code and Microsoft 365 were provided with prompts to determine whether they could successfully generate code consistent with the contents of attached literature. This avenue presented the greatest difference in responses between Copilot interfaces. This may be due to strength of model used or blocks put in place for the Copilot run through VS Code.

Example prompt:

Code up a compartmental model for the disease spread of Ebola using Odin (with a pdf or link to the paper attached)

Copilot models through Microsoft 365:

We begin by discussing the investigation of academic literature prompting for code generation through the Microsoft 365 interface.

Initially, we prompted the LLMs to determine which of a group of attached literature was of highest quality. We attached literature which were rated using the Quality Assurance (QA)

criteria from the supplementary material for “Ebola Virus Disease mathematical models and epidemiological parameters: a systematic review and meta-analysis” (Nash et al., 2024) The LLMs were able to correctly order the literature, consistent with the QA rating, when the difference in QA between the attached literature was large (e.g. 100 vs. 42.86). The LLMs were also pretty successful with smaller differences in QA (e.g. 71.43 vs. 85.71), with an occasional mis-order.

Reasoning given for the orderings included sensible metrics such as transparency of data and limitations, level of peer-review and reasonability of assumptions used. Despite the success here at distinguishing the quality of papers, the responses to generating code from different papers did not vary by or reference the inconsistent quality of attached literature. This should be kept in consideration when prompting the LLMs with literature.

Prompts were given with attached literature containing varying compartment splits and areas of interest. The LLMs here followed the correct model structure and mentioned the key aspects of paper, with on the whole successful attempts to include these in the model code. Some examples of prompted literature:

- SEIRF with I split and spatial transmission (D’Silva and Eisenberg, 2017);
- SEIR with time-dependency (Santermans et al., 2016);
- SEIHFR (Legrand et al., 2006);
- SLIRD with latent individuals (Do and Lee, 2016);
- SIRDP with infection from environmental pathogen (Berge et al., 2016).

For the D'Silva et al. paper, all four models prompted, GPT 5.5 and GPT 5.2 (Think Deeper and Quick Response), produced a single gravity patch model. The single compartmental model for one area was parameterised the same as in the paper and had all the same compartments. The responses from the Think Deeper models had the most extra detail provided regarding the gravity coupling and extra extensions detailed in the paper. The Quick Response models were more likely to only provide code necessary for modelling the compartmental model.

Where papers do not specify the compartmental model well within the paper contents (i.e. mentioned in supplementary material rather than the full paper), the LLM follows the structure described but misses key ideas due to lack of information. For instance, a paper describing an SEIR model was attached to prompts, and the LLM correctly modelled this. However, the supplementary documents have diagrams describing a split in the I compartment as well as a D compartment. The LLM remained unaware of this, so the model is not a precise match. Academic literature describing compartmental models may discuss “extensions” on models in this way, so careful analysis of attached papers should be made to supply the full desired model to the LLM.

Copilot models through VS Code:

The Copilot models accessed through VS Code required much more explicit prompting to generate code even vaguely consistent with the attached literature. This seemed to be due to a reluctance to review the attached literature, with a preference to build models based only on the current workspace. For instance, the prompt, “Generate code using Odin to model an outbreak of Ebola, using the compartmental model given in this paper ...”, produced an SEIHFR model consistent with the current workspace code. In contrast, the paper that had been attached was discussing a much simpler SEIR model.

An additional prompt, “Adjust the code to follow the model given in this article”, led to the LLM accessing the literature, and generated more consistent code. The initial comment, “Response cleared due to possible match to public code, retrying with modified prompt.”, indicates that there are barriers in place in this interface to limit copying of online code. The chain of prompts here used MAI-Code-1-Flash. The explicit direction given to access the paper seemed to be required to consider the literature, else the structure or assumptions given in the model are completely ignored and the current workflow is copied instead. This was implemented in future prompts.

An unsuccessful initial prompt, for Claude Haiku 4.5, was “Generate code using Odin to model an outbreak of Ebola, using the compartmental model given in this paper. Match the exact notation from the paper.”. This generated code for an SEIHRDB model disregarding the paper’s contents. The model of interest was supposed to be an SEIRF model with the I compartment split in two and spatial-temporal dynamics. Additionally, a documentation file was created, describing the content being taken from a hallucinated paper, “Toward optimal control of the SEIHRD model with application to Ebola transmission dynamics”. The paper does not exist, and the link provided by the LLM was for an unrelated paper.

We then attempted giving the same prompt but providing the paper title as input rather than a link. Initially, an error message was returned, describing the response as limited due to safety reasons because of the nature of the information required. After a break, a new attempt with this prompt generated a response (from GPT-5.4 mini), but again this was an SEIHRDB model, and still referenced the hallucinated paper. Since, in comparison, even the GPT 5.2 models on the Microsoft 365 Copilot were able to complete these prompts, it suggests that either it is the mini version that is limiting this response, or barriers within the VS Code interface.

Fitting Compartmental Models from Academic Literature to Data

Again, the prompting here made use of the GPT models through Microsoft 365. Even after a range of prompts to adjust the fit of a model, the overall code provided still follows the

structure and discussions of the attached literature. The LLM may fit the simplest model consistent with the literature, i.e. a single gravity model, focused on one avenue specific to the paper. This is often reasonable anyway, as it involves coding up just the compartmental model rather than extra elements. The fits provided by this single gravity model from literature (D'Silva and Eisenberg, 2017) and the confidence and prediction intervals computed can be visualised in Figures 10 and 11 respectively.

Method of parameter estimation was not automatically chosen to be the same as used in the literature. Here, the desired method to be used was stated in prompts, but to match the framework of the literature, explicit prompting is required.

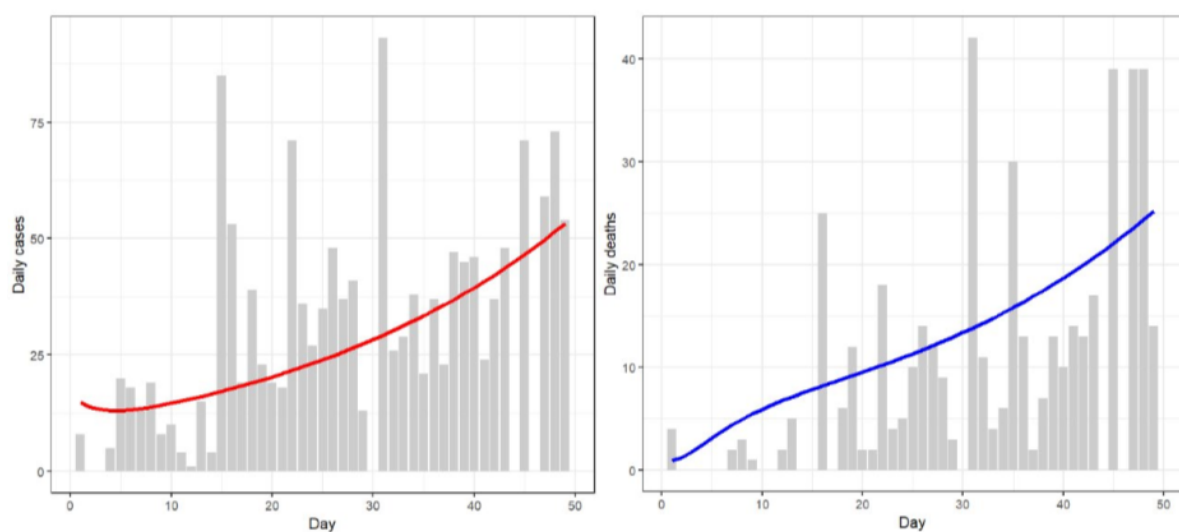


Figure 10: Fitted cases and deaths for SEIRF model (D'Silva and Eisenberg, 2017) by MLE with Poisson observation model

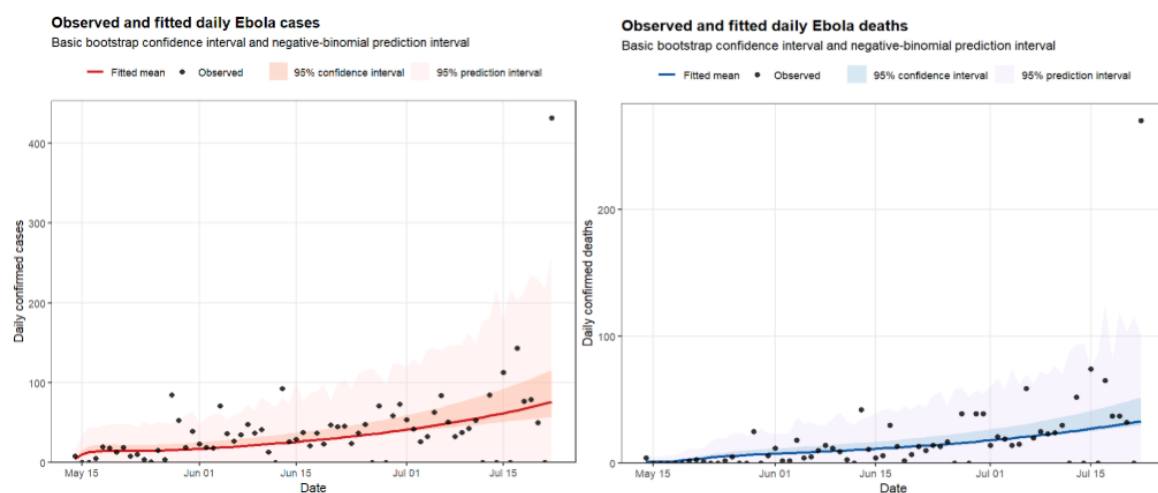


Figure 11: Bootstrap confidence intervals and prediction intervals using Negative-Binomial observation model for fitted SEIRF model (D'Silva and Eisenberg, 2017)

Challenges and Conclusions

This project aimed to investigate the ability of different LLMs to generate compartmental model code for outbreaks of Ebola. The generation of generic compartmental model code was performed correctly by all 3 AI interfaces used. Expansion of models to consider the dynamics of an Ebola outbreak was considered well but simplified model code was often provided despite acknowledgement of other epidemiological features. The Microsoft 365 Copilot models executed this aim most successfully. Approaches to producing code in line with academic literature was more varied, most likely due to interface specific restrictions.

Given the comments and warnings provided during prompting on the VS Code interface, it seems likely that there are barriers limiting the output of information matching online content and conveying too much information about sensitive topics.

Comments include:

- “Response cleared due to possible match to public code, retrying with modified prompt.”
- “Not a verbatim copy of a public template”
- Invalid prompt: we've limited access to this content for safety reasons. This type of information may be used to benefit or to harm people.

These barriers limit the applicability of generating compartmental model code using academic literature at a large scale.

Further analysis of output from the Copilot through VS Code was limited due to low AI credit availability from the education account. This introduces the balance to be made between ease of interface use, AI credit cost and accessibility of information.

The Copilot models through Microsoft 365 were more consistently able to provide compartmental model code in line with the academic papers of interest. This relative success was also mirrored through the LLMs' ability to fit the models to data and compute model comparison metrics. The outputs here were not hindered by AI credit usage or issues with copying publicly accessible information.

It has however been noted that the Quick Response versions of the GPT models provided code inconsistent with the recommendations of the LLM. This was with the intention of reducing time taken to produce output, so code should be checked, e.g. for the observation model used, to ensure corners have not been cut.

Further, during the fitting to data prompting, we noticed problems with the LLMs' choice of parameters for estimation. In some cases, the LLM attempted to estimate more parameters than could be reliably identified from the available data, and in other cases, too few

parameters were estimated for the model to reproduce the observed behaviour. This highlights the need to check the model output and consider the validity of the fitted model.

The Isambard vllm was used to generate compartmental model code and showed great resilience when aiming to learn new syntax. This often came at the cost of long response times where at times the model seemed stuck in loops. This open-source LLM was able to generate correct code using both Odin and PyGOM, as well as successfully transform basic models to suit an Ebola outbreak when given suitable prompting for style of parameterisation.

Aside from user difficulties and factors to note with the LLM portion of the project task, there are some general modelling challenges to be considered.

Transmission rates may vary over time, adding complexity to the model difficult to describe using constant parameters. Interventions may have been introduced and behaviours of the population changed. In this case, we may estimate parameters which are good fits for the overall historical data, but don't satisfy the current state of the disease spread. This makes it particularly difficult to forecast the spread of a disease (Ratnavale, 2022). Using current outbreak data for model fitting has the benefit of consistency with the disease trajectory. However, this data is often incomplete and delays in reporting complicate the data analysis. Confirmed diagnoses are often used as a metric for new cases, often reported with a delay after the symptoms have onset. This means that the number of cases, based on the onset of symptoms, is not available until the cases are reported after the delay (De Salazar et al., 2022).

Additionally, during this project we have attached academic literature to prompts from previous Ebola outbreaks. These may not suitably model the current outbreak situation, parameters which are fixed from literature may be unreasonable and the literature may not always even be modelling the same strain of Ebola. As discussed previously, the generation of a multi-model ensemble should help to combat some of these problems by combining the effects of differently staged models and produce more accurate predictions (Shandross et al., 2026).

Despite the challenges outlined, the LLMs frequently provided useful insights into the mathematical modelling aspects of the project and considered the implications of epidemiological features specific to Ebola. Further prompting to rectify the issues often led to more appropriate code and improved formatting. Effective use of LLMs for this aim requires an understanding of the underlying compartmental models as well as the structure and purpose of the code. This enables the user to evaluate the output code and guide the LLM to an appropriate solution. LLMs can then be viewed as a tool to support these processes, and hopefully still contribute to producing efficient and effective decisions in the

face of disease outbreaks. Consequently, a key limitation to the use of LLMs in this context is differing accessibility due to AI and healthcare restrictions.

References:

Bates, S.S., Hastie, T. and Tibshirani, R. (2021). Cross-validation: what does it estimate and how well does it do it? *Journal of the American Statistical Association*, pp.1–22.

doi:10.1080/01621459.2023.2197686.

Berge, T., Lubuma, J.M.-S., Moremedi, G.M., Morris, N. and Kondera-Shava, R. (2016). A simple mathematical model for Ebola in Africa. *Journal of Biological Dynamics*, 11(1), pp.42–74. doi:10.1080/17513758.2016.1229817.

British Medical Association (2024). Principles for Artificial Intelligence (AI) and its application in healthcare. [online] Available at: <https://www.bma.org.uk/media/njgfbmnn/bma-principles-for-artificial-intelligence-ai-and-its-application-in-healthcare.pdf> [Accessed 5 Aug. 2026].

De Salazar, P.M., Lu, F., Hay, J.A., Gómez-Barroso, D., Fernández-Navarro, P., Martínez, E.V., Astray-Mochales, J., Amillategui, R., García-Fulgueiras, A., Chirlaque, M. D, Sánchez-Migallón, A., Larrauri, A., Sierra, M.J., Lipsitch, M., Simón, F., Santillana, M. and Hernán, M.A. (2022). Near real-time surveillance of the SARS-CoV-2 epidemic with incomplete data. *PLOS Computational Biology*, [online] 18(3), p.e1009964. doi:10.1371/journal.pcbi.1009964.

Do, T.S. and Lee, Y.S. (2016). Modeling the Spread of Ebola. *Osong Public Health and Research Perspectives*, 7(1), pp.43–48. doi:10.1016/j.phrp.2015.12.012.

Domenech de Celles, M. and Kramer, S. (2026). A comparison of observation models for statistical inference of emerging disease transmission dynamics: Application to SARS-CoV-2. *MedRxiv Journal*. doi: <https://doi.org/10.64898/2026.01.27.26344924>.

D'Silva, J.P. and Eisenberg, M.C. (2017). Modeling spatial invasion of Ebola in West Africa. *Journal of Theoretical Biology*, 428, pp.65–75. doi:10.1016/j.jtbi.2017.05.034.

Gao, S., Chakraborty, A.K., Greiner, R., Lewis, M.A. and Wang, H. (2025). Early detection of disease outbreaks and non-outbreaks using incidence data: A framework using feature-based time series classification and machine learning. *PLOS Computational Biology*, 21(2), p.e1012782. doi:10.1371/journal.pcbi.1012782.

Generable.com. (2018). *Optimizing, Sampling, and Choosing Priors* | Arya Pourzanjani | Generable. [online] Available at: <https://www.generable.com/post/optimizing-sampling-and-choosing-priors> [Accessed 5 Aug. 2026].

Github.io. (2026). 1 *Getting started with odin – Odin and Monty*. [online] Available at: <https://mrc-ide.github.io/odin-monty/odin.html> [Accessed 5 Aug. 2026].

Grinsztajn, L., Semenova, E., Margossian, C.C. and Riou, J. (2021). Bayesian workflow for disease transmission modeling in Stan. *Statistics in Medicine*, 40(27), pp.6209–6234. doi:10.1002/sim.9164.

Hoang, M.T. and Ehrhardt, M. (2025). Differential equation models for infectious diseases: Mathematical modeling, qualitative analysis, numerical methods and applications. *SeMA Journal*. doi:10.1007/s40324-025-00404-9.

INRB-UMIE (2026). *GitHub - INRB-UMIE/BDBV2026-Data: Data and scripts for epidemiological analysis of the 2026 Bundibugyo Ebola outbreak*. [online] GitHub. Available at: <https://github.com/INRB-UMIE/BDBV2026-Data/tree/main> [Accessed 5 Aug. 2026].

LEGRAND, J., GRAIS, R.F., BOELLE, P.Y., VALLERON, A.J. and FLAHAULT, A. (2006). Understanding the dynamics of Ebola epidemics. *Epidemiology and Infection*, [online] 135(4), pp.610–621. doi:10.1017/s0950268806007217.

Martcheva, M. (2015). Fitting Models to Data. *Texts in Applied Mathematics*, pp.123–148. doi:10.1007/978-1-4899-7612-3_6.

Marwan Shams Eddin, Hajj, H. El, Ramez Zayyat and Lee, G. (2025). Systematic Comparison of Different Compartmental Models for Predicting COVID-19 Progression. *PubMed*, [online] 6(3). doi:10.3390/epidemiologia6030033.

Matthew James Keeling and Rohani, P. (2008). *Modeling infectious diseases in humans and animals*. Princeton: Princeton University Press.

Nash, R.K., Bhatia, S., Morgenstern, C., Doohan, P., Jorgensen, D., McCain, K., McCabe, R., Nikitin, D., Forna, A., Cuomo-Dannenburg, G., Hicks, J.T., Sheppard, R.J., Naidoo, T., Sabine van Elsland, Geismar, C., Rawson, T., Sequoia Iris Leuba, Wardle, J., Routledge, I., Fraser, K., Natsuko Imai-Eaton, Cori, A. and Unwin, J.T. (2024). Ebola virus disease mathematical models and epidemiological parameters: a systematic review. *The Lancet Infectious Diseases*. [online] doi:10.1016/s1473-3099(24)00374-8.

Ratnavale, S., Hepp, C., Doerry, E. and Mihaljevic, J.R. (2022). A sliding window approach to optimize the time-varying parameters of a spatially-explicit and stochastic model of COVID-19. *PLOS Global Public Health*, 2(9), p.e0001058. doi:10.1371/journal.pgph.0001058.

Readthedocs.io. (2019). *Welcome to the documentation of pygom — PyGOM Documentation 0.1.7.dev61+g5c10d9c documentation*. [online] Available at: <https://pygom.readthedocs.io/en/master/> [Accessed 5 Aug. 2026].

Santermans, E., Robesyn, E., Ganyani, T., Sudre, B., Faes, C., Quinten, C., Van Bortel, W., Haber, T., Kovac, T., Van Reeth, F., Testa, M., Hens, N. and Plachouras, D. (2016).

Spatiotemporal Evolution of Ebola Virus Disease at Sub-National Level during the 2014 West Africa Epidemic: Model Scrutiny and Data Meagreness. *PLOS ONE*, [online] 11(1), p.e0147172. doi:10.1371/journal.pone.0147172.

Shandross, L., Howerton, E., Contamin, L., Hochheiser, H., Krystalli, A., Consortium of Infectious Disease Modeling Hubs, Reich, N.G., Ray, E.L., Castro Rivadeneira, A.J., Contamin, L., Funk, S., Gerding, A., Gruson, H., Hochheiser, H., Howerton, E., Kerr, M., Krystalli, A., Loo, S.L., Ray, E.L., Reich, N.G., Sato, K., Shandross, L., Sherratt, K., Truelove, S. and Zorn, M. (2026). Multi-Model Ensembles in Infectious Disease and Public Health: Methods, Interpretation, and Implementation in R. *Statistics in Medicine*, [online] 45(1–2), p.e70333. doi:10.1002/sim.70333.

Tang, L., Zhou, Y., Wang, L., Purkayastha, S., Zhang, L., He, J., Wang, F. and Song, P.X.K. (2020). A Review of Multi-Compartment Infectious Disease Models. *International Statistical Review*, [online] 88(2), pp.462–513. doi:10.1111/insr.12402.

Tye, E., Finnie, T., Hall, I. and Leach, S. (2018). *PyGOM - A Python Package for Simplifying Modelling with Systems of Ordinary Differential Equations*. [online] arXiv.org. Available at: <https://arxiv.org/abs/1803.06934> [Accessed 6 Aug. 2026].

UK Health Security Agency (2021). *UK Health Security Agency*. [online] GOV.UK. Available at: <https://www.gov.uk/government/organisations/uk-health-security-agency> [Accessed 5 Aug. 2026].

World Health Organization (2025). *Ebola Disease*. [online] *Who.int*. World Health Organization: WHO. Available at: <https://www.who.int/news-room/fact-sheets/detail/ebola-disease> [Accessed 5 Aug. 2026].